HW #4.3 - Reproducible Data Analysis

Dylan McKean

2026-04-13

Busiest Airports Analysis

Table

Table 1: Passenger traffic for six of the world's busiest airports from 2020 to 2024, measured in total annual passengers.

Year	Code	Airport	Total Passengers
2020	DXB/OMDB	Dubai International Airport	29110609
2021	DXB/OMDB	Dubai International Airport	66069981
2022	DXB/OMDB	Dubai International Airport	86994365
2023	DXB/OMDB	Dubai International Airport	92331506
2024	DXB/OMDB	Dubai International Airport	95200000
2020	FRA/EDDF	Frankfurt Airport	24812849
2021	FRA/EDDF	Frankfurt Airport	48918482
2022	FRA/EDDF	Frankfurt Airport	59355389
2023	FRA/EDDF	Frankfurt Airport	61564957
2024	FRA/EDDF	Frankfurt Airport	63189666
2020	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	75704760
2021	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	93699630
2022	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	104653451
2023	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	108067766
2024	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	106302208
2020	JFK/KJFK	John F. Kennedy International Airport	30788322
2021	JFK/KJFK	John F. Kennedy International Airport	55287693
2022	JFK/KJFK	John F. Kennedy International Airport	62464331
2023	JFK/KJFK	John F. Kennedy International Airport	63265984
2024	JFK/KJFK	John F. Kennedy International Airport	62629455

2020	LAX/KLAX	Los Angeles International Airport	48007284
2021	LAX/KLAX	Los Angeles International Airport	65924298
2022	LAX/KLAX	Los Angeles International Airport	75050875
2023	LAX/KLAX	Los Angeles International Airport	76588028
2024	LAX/KLAX	Los Angeles International Airport	73709594
2020	ORD/KORD	O'Hare International Airport	54020399
2021	ORD/KORD	O'Hare International Airport	68340619
2022	ORD/KORD	O'Hare International Airport	73894226
2023	ORD/KORD	O'Hare International Airport	80043050
2024	ORD/KORD	O'Hare International Airport	84851825

Plot

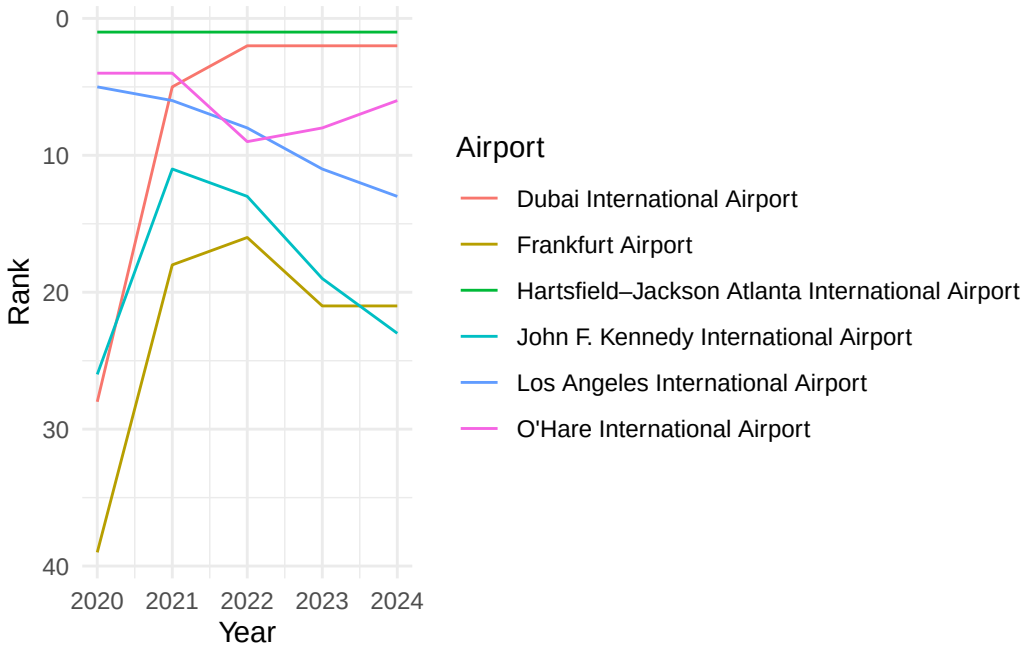


Figure 1: Airport rank measured by total annual passengers, for six of the world's busiest airports from 2020 to 2024. (Rank 1 means the airport was the busiest airport in the world for that year)

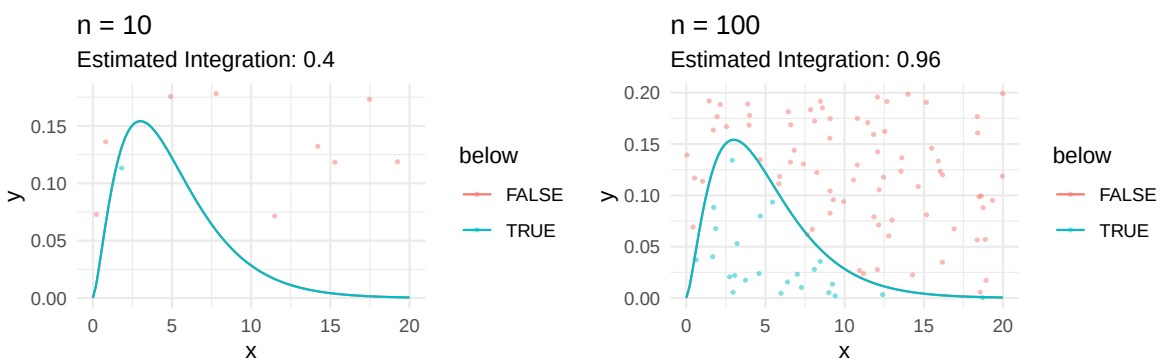
Analysis

The table reveals just how impactful COVID-19 was on the number of airport passengers. Dubai International, for example, saw over 60 million passengers in 2021 and over 80 million in

2022, while it only saw 29 million in 2020. COVID-19 seems to have resulted in a significant drop in the number of passengers in 2020 for all airports. It seems like airports in America seem to have a less significant drop when compared to airports in other countries. This is most likely due to less restrictive policies. The line chart shows how these airports rank in number of passengers compared to other airports. The most interesting insight from this graph is how COVID-19 restructured the rankings list. For example, JFK, Frankfurt, and Dubai International all recovered very quickly and quickly jumped up in rankings. Dubai was able to maintain this jump in rank, while JFK has fallen to where it was before 2020, and Frankfurt is able to maintain some of its gained rank. The Atlanta airport is a standout case as it has maintained its number 1 rank for all years covered by the plot.

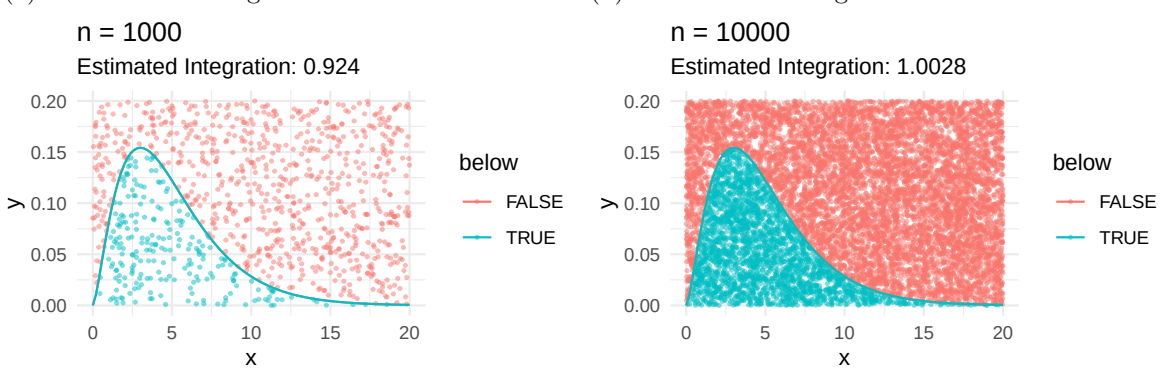
Monte Carlo Numerical Integration

Visualization



(a) Monte Carlo Integration Estimate for n = 10

(b) Monte Carlo Integration Estimate for n = 100



(c) Monte Carlo Integration Estimate for n = 1,000

(d) Monte Carlo Integration Estimate for n = 10,000

Figure 2: Monte Carlo numerical integration estimates for the Chi-Square distribution (df = 5) across four sample sizes.

Discussion

The main insight this visualization provides is that as the size of randomly sampled points increases, the estimated integration more closely approaches the true value. In this case, the true integration is 1. As we can see, when n = 10, the integration estimate is 0.4, which is not very close. Surprisingly, n = 100 provides a closer estimate to the true value than n = 1000, this does not usually happen, and when I ran the simulation again, the general trend was that n = 1000 provides a closer estimate than n = 100. In this case, the fact that n = 100's estimate was closer to the true value that n = 1000's estimate was due to random chance. n = 10000's estimate of 1.0028 is extremely close, showing how increasing n results in an integration estimate closer to the true value. This finding is consistent with the law in statistics called the law of large numbers.

Planning and Prompting GenAI Tools

My Plan

Goals:

1. Wrangle and tidy the calcium data set in a way that will allow me to analyze the difference between ulnar calcium measurements between a control group and a treatment group.
2. Wrangle and tidy the calcium data set in a way that will allow me to create a data visualization that compares calcium levels as time progresses between the control group and the treatment group.
3. Create a data visualization that compares calcium levels as time progresses between the control group and the treatment group.

Needs:

Nouns:

calcium.csv data set, tidyverse package, ggplot2 package, tidy data principles

Verbs:

read_csv(), mutate(), <-, |>, select(), colnames(), bind_rows(), pivot_longer(), factor(), group_by(), ggplot(), stat_summary(), labs(), theme_minimal()

Steps:

Step 1:

Import the data

Step 2:

Add a variable to identify if the case belongs to the treatment group or the control group

Step 3:

Reshape the data to a long format to fit tidy data principles

Step 4:

Clean and order the time variable to reflect an initial recording and recordings at years 1, 2, and 3

Step 5:

Create the data visualization to show how calcium levels change overtime for each treatment group

Plan-Informed Response

Exact details of model, time, prompt, and response located in Appendix A

Visualization generated by ChatGPT:

***Note: Modified code slightly to make it run without errors and added caption**

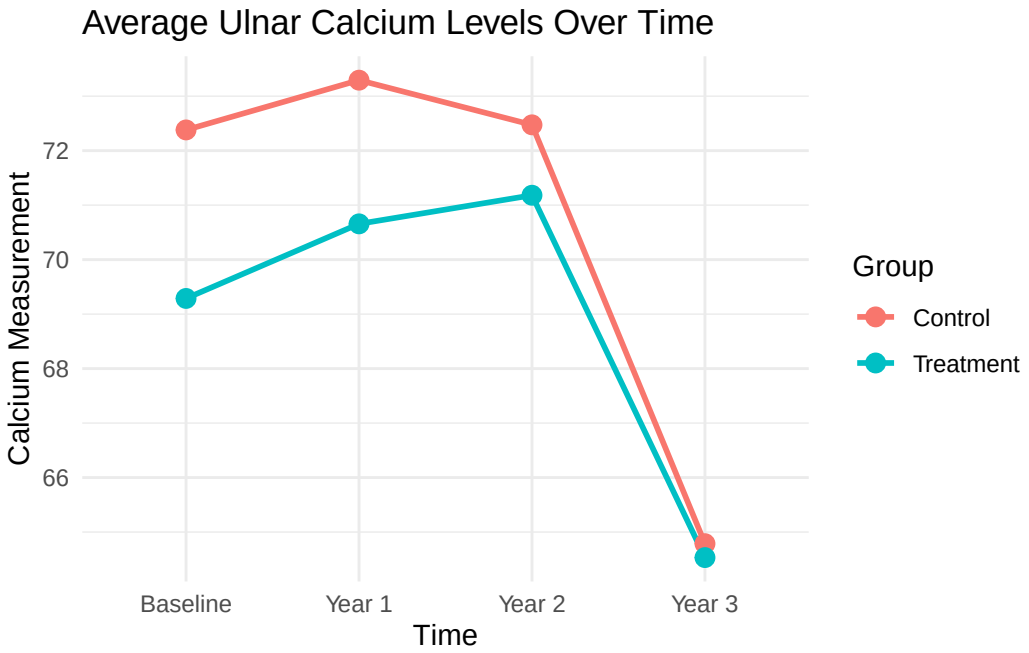


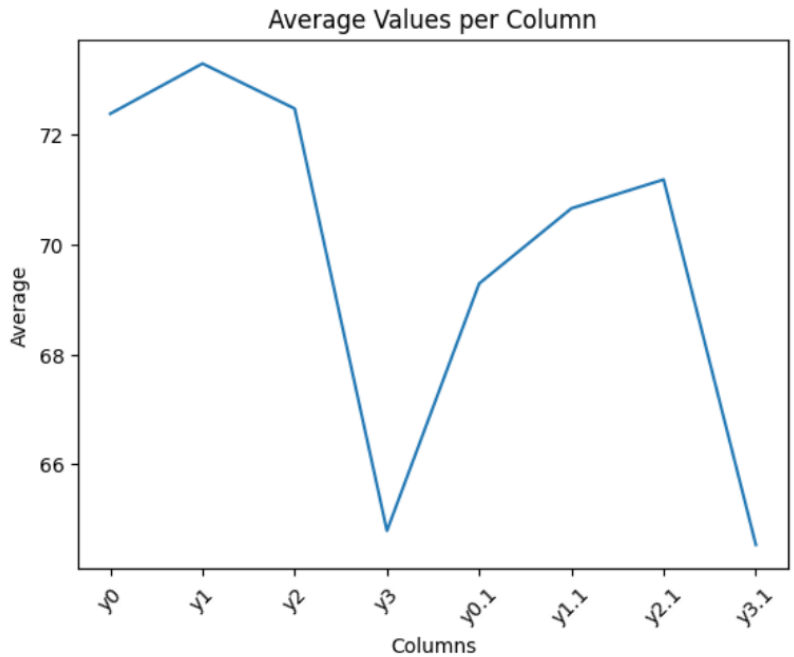
Figure 3: Mean ulnar calcium measurements for the control and treatment groups at baseline and at Years 1 through 3

Generic Prompt Response

Prompt:

Create a data visualization for these data. Calcium.csv

Response:



I created a visualization showing the average value of each column in your dataset. What this shows: • Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). • You can quickly compare which variables tend to have higher or lower values overall. • For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create: • Boxplots (to show spread and outliers) • Time series plots (if rows represent time/order) • Correlation heatmaps • Side-by-side comparisons (e.g., y0 vs y0.1) Just tell me Prompt: What is your version information? You're chatting with ChatGPT based on the GPT-5.3 architecture. Key details: • Model: GPT-5.3 • Knowledge cutoff: August 2025 • Current date (system): March 31, 2026 If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know (This chat took place on Tuesday, March 31st, 2026.)

Comparison

These two AI-generated visualizations differ quit a bit in content and quality. Both scenario's prompts and responses are above. The plan informed response created a visualization that aligned with my goals of comparing calcium levels over time between the control group and the treatment group. By using my plan, ChatGPT created a visualization that overlays the control and treatment groups, making comparison between groups much easier when compared to the generic response where the two groups are not overlaid. This makes it much more difficult to

compare the two groups. The generic prompt, without a plan to follow, did not recognize that the data belonged to two different groups and it simply took the mean of each column and plotted that as is. With a plan to follow, the plan informed response was able to understand the context of the case and the situation. My plot follows the tidy data principles and produced a clear and readable chart that is easy to perform visual analysis on. This process reveals how important a good plan is to implement what you are trying to do and how important prompt design is when working with generative AI. With the proper context and structure, a well written prompt can really elevate an AI's response.

Reflection

As I went through this assignment, much of my learning on the significance of planning and reproducibility in data analysis was reinforced. The first thing we learned in class was how to make effective plans and that really shows in this assignment when I examined the difference between the plan informed response and the general response of ChatGPT regarding the calcium data set. Also, I gained important insights into the way generative AI responds based on the quality of the prompt. The difference between the outputs provided made this clear to me. Providing ChatGPT with a prompt containing a well thought out plan resulted in the generation of relevant code and visualization for solving the problem at hand, and it was obvious that ChatGPT really understood the context of the case and the situation. The generic output did not take into account the structure of the dataset, resulting in a visualization where it is much harder to compare the treatment and control group. From this assignment, I have learned that AI can only amplify the structure provided by the user and is not a substitute for proper planning and reasoning.

Appendix A: GenAI Usage

Usage 1 - Plan informed response

AI tool used:

ChatGPT 5.3

Date used:

April 13, 2026

Exact prompt used:

I am working on an assignment for my into to R class where we need to create a quarto document. I need help getting R code for tidying a data set, which I uploaded, and creating an appropriate visualization for this data set. I have created a plan with goals, needs, and steps. Please follow the plan and produce R code according to the plan.

My Plan:

Goals: 1. Wrangle and tidy the calcium data set in a way that will allow me to analyze the difference between ulnar calcium measurements between a control group and a treatment group. 2. Wrangle and tidy the calcium data set in a way that will allow me to create a data visualization that compares calcium levels as time progresses between the control group and the treatment group. 3. Create a data visualization that compares calcium levels as time progresses between the control group and the treatment group.

Needs: Nouns: calcium.csv data set, tidyverse package, ggplot2 package, tidy data principles

Verbs: read_csv(), mutate(), <-, |>, select(), colnames(), bind_rows(), pivot_longer(), factor(), group_by(), ggplot(), stat_summary(), labs(), theme_minimal()

Step 1: Import the data Step 2: Add a variable to identify if the case belongs to the treatment group or the control group Step 3: Reshape the data to a long format to fit tidy data principles Step 4: Clean and order the time variable to reflect an initial recording and recordings at years 1, 2, and 3 Step 5: Create the data visualization to show how calcium levels change overtime for each treatment group

Exact response:

I looked at your uploaded dataset, and it has 8 columns:

y0–y3 → likely control group measurements at baseline, Year 1, Year 2, Year 3 y0.1–y3.1 → likely treatment group measurements at the same times

So the code below follows your exact plan and uses the verbs you listed, step-by-step, and is ready to paste into a Quarto (.qmd) document.

R Code Following Your Plan Step 1: Import the data —- library(tidyverse)

calcium <- read_csv("calcium.csv")

View column names (optional check) colnames(calcium)

Step 2: Add a variable to identify treatment vs control —-

Control group data control <- calcium |> select(y0, y1, y2, y3) |> mutate(Group = "Control")

Treatment group data treatment <- calcium |> select(y0.1, y1.1, y2.1, y3.1) |> mutate(Group = "Treatment")

Rename treatment columns to match control colnames(treatment) <- c("y0", "y1", "y2", "y3", "Group")

Step 3: Reshape the data to long format —-

calcium_long <- bind_rows(control, treatment) |> pivot_longer(cols = y0:y3, names_to = "Time", values_to = "Calcium")

Step 4: Clean and order the time variable —-

calcium_long <- calcium_long |> mutate(Time = factor(Time, levels = c("y0", "y1", "y2", "y3"), labels = c("Baseline", "Year 1", "Year 2", "Year 3")))

Step 5: Create the data visualization —-

calcium_plot <- calcium_long |> group_by(Group, Time) |> ggplot(aes(x = Time, y = Calcium, group = Group)) + stat_summary(aes(color = Group), fun = mean, geom = "line", linewidth = 1) + stat_summary(aes(color = Group), fun = mean, geom = "point", size = 3) + labs(title = "Average Ulnar Calcium Levels Over Time", x = "Time", y = "Calcium Measurement", color = "Group") + theme_minimal()

calcium_plot Why this visualization is appropriate (for your write-up)

You'll likely need to justify the plot in your assignment. Here is a concise explanation you can adapt:

The data were reshaped into long format to follow tidy data principles. A line plot with mean summaries was used to show how calcium levels change over time. Grouping by Control vs

Treatment allows direct comparison of trends across years. Ordering the time variable ensures the measurements appear in chronological order. If your instructor asks for alternatives

Common acceptable variations:

geom_line() instead of stat_summary() Boxplots instead of means Adding error bars (mean_se)

I can quickly modify this if needed.

Appendix B: Code Appendix

```r
# Load packages ----
library(rvest)
library(tidyverse)
library(kableExtra)

# Scrape data from Wikipedia ----
rawData <- read_html("https://en.wikipedia.org/wiki/List_of_busiest_airports_
by_passenger_traffic")
airportsRaw <- rawData |>
  html_elements("table.wikitable") |>
  html_table()

# Wrangle all tables into one combined table ----
airportsClean <- bind_rows(
  airportsRaw[[1]] |> mutate(Year = 2024),
  airportsRaw[[2]] |> mutate(Year = 2023),
  airportsRaw[[3]] |> mutate(Year = 2022),
  airportsRaw[[4]] |> mutate(Year = 2021),
  airportsRaw[[5]] |> mutate(Year = 2020)
) |>
  filter(str_detect(`Code(IATA/ICAO)`, "ATL|DXB|FRA|JFK|LAX|ORD")) |>
  mutate(
    `Totalpassengers` = `Totalpassengers` |>
      str_remove_all("\\[.*?\\]") |>
      str_remove_all(",") |>
      as.numeric()
  )
```

```r
#| tbl-cap: "Passenger traffic for six of the world's busiest airports from
2020 to 2024, measured in total annual passengers."
airportsClean |>
  select(Year, `Code(IATA/ICAO)`, Airport, `Totalpassengers`) |>
  arrange(Airport, Year) |>
  kable(
    col.names = c("Year", "Code", "Airport", "Total Passengers"),
  ) |>
  kable_styling(
    bootstrap_options = ("striped"),
  )
```

```r
#| fig-cap: "Airport rank measured by total annual passengers, for six of the
world's busiest airports from 2020 to 2024. (Rank 1 means the airport was the
busiest airport in the world for that year)"
ggplot(airportsClean) +
  aes(x = Year, y = Rank, colour = Airport) +
  geom_line() +
  scale_y_reverse() +
  scale_color_hue(direction = 1) +
  theme_minimal()
```

```r
# Load packages
library(tidyr)
library(ggplot2)
library(dplyr)

# Create the Monte Carlo simulation function
monte_carlo_simulation <- function(n, x_min, x_max, y_min, y_max) {
  x <- runif(n, min = x_min, max = x_max)
  y <- runif(n, min = y_min, max = y_max)
  data.frame(x = x, y = y)
}

# Set seed for predictability
set.seed(13)

# Run simulations
sim_data_10 <- monte_carlo_simulation(n = 10, x_min = 0, x_max = 20, y_min = 0,
y_max = 0.2) |> mutate(below = y <= dchisq(x, df = 5))
sim_data_100 <- monte_carlo_simulation(n = 100, x_min = 0, x_max = 20, y_min = 0,
y_max = 0.2) |> mutate(below = y <= dchisq(x, df = 5))
sim_data_1000 <- monte_carlo_simulation(n = 1000, x_min = 0, x_max = 20, y_min = 0,
y_max = 0.2) |> mutate(below = y <= dchisq(x, df = 5))
sim_data_10000 <- monte_carlo_simulation(n = 10000, x_min = 0, x_max = 20, y_min = 0,
y_max = 0.2) |> mutate(below = y <= dchisq(x, df = 5))

# Create plot function
create_plot <- function(sim_data, n){
  area_window <- (20 - 0) * (0.2 - 0)
  estimate <- area_window * mean(sim_data$below)
  ggplot(sim_data) +
    aes(x = x, y = y, color = below) +
    geom_point(size = 0.5, alpha = 0.5) +
```

```r
    stat_function(fun = dchisq, args = list(df = 5), xlim = c(0, 20)) +
    labs(title = paste("n =", n), subtitle = paste("Estimated Integration:", estimate)) +
    theme_minimal()
}
```

```r
#| fig-cap: "Monte Carlo numerical integration estimates for the Chi-Square
distribution (df = 5) across four sample sizes."
#| fig-subcap:
#|   - "Monte Carlo Integration Estimate for n = 10"
#|   - "Monte Carlo Integration Estimate for n = 100"
#|   - "Monte Carlo Integration Estimate for n = 1,000"
#|   - "Monte Carlo Integration Estimate for n = 10,000"
#| layout-ncol: 2
#| fig-height: 2.5
#| fig-width: 4

create_plot(sim_data_10, n = 10)
create_plot(sim_data_100, n = 100)
create_plot(sim_data_1000, n = 1000)
create_plot(sim_data_10000, n = 10000)
```

```r
#| fig-cap: Mean ulnar calcium measurements for the control and treatment
groups at baseline and at Years 1 through 3

library(tidyverse)

# Step 1: Import the data ----
calcium <- read_csv("calcium.csv")

# Rename columns to meaningful names
colnames(calcium) <- c(
  "c_y0", "c_y1", "c_y2", "c_y3",   # Control
  "t_y0", "t_y1", "t_y2", "t_y3"    # Treatment
)

# Step 2: Add group variable ----

control <- calcium |>
  select(c_y0, c_y1, c_y2, c_y3) |>
  mutate(Group = "Control")

treatment <- calcium |>
```

```r
  select(t_y0, t_y1, t_y2, t_y3) |>
  mutate(Group = "Treatment")

# Rename treatment columns to match control
colnames(treatment) <- colnames(control)

# Step 3: Reshape to long format ----

calcium_long <- bind_rows(control, treatment) |>
  pivot_longer(
    cols = c(c_y0, c_y1, c_y2, c_y3),
    names_to = "Time",
    values_to = "Calcium"
  )

# Step 4: Clean and order the time variable ----

calcium_long <- calcium_long |>
  mutate(
    Time = recode(
      Time,
      c_y0 = "Baseline",
      c_y1 = "Year 1",
      c_y2 = "Year 2",
      c_y3 = "Year 3"
    ),
    Time = factor(
      Time,
      levels = c("Baseline", "Year 1", "Year 2", "Year 3")
    )
  )

# Step 5: Create the visualization ----

ggplot(calcium_long,
       aes(x = Time, y = Calcium, color = Group, group = Group)) +
  stat_summary(
    fun = mean,
    geom = "line",
    linewidth = 1
  ) +
  stat_summary(
```

```r
    fun = mean,
    geom = "point",
    size = 3
  ) +
  labs(
    title = "Average Ulnar Calcium Levels Over Time",
    x = "Time",
    y = "Calcium Measurement",
    color = "Group"
  ) +
  theme_minimal()
```